

11 计算机系统概述

计算机的历史：机械计算机、真空管、晶体管、集成电路、大规模集成电路与超大规模集成电路

图灵机：

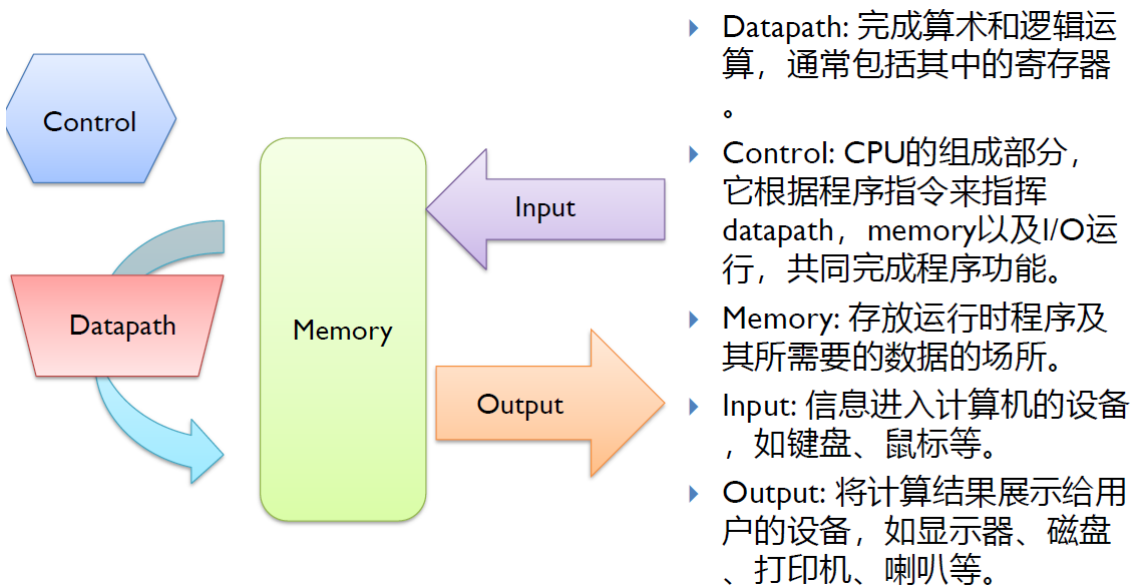
- **存储程序**的思想，使计算机从专用→**通用**（执行一个描述好的程序）
- **通用计算机**确定了现代计算机的理论基础
- 图灵机的特点除上述两个之外还有有限速度、有限空间

1946年第一台通用电子计算机 ENIAC，之后第一台VonNeumann计算机 EDSAC，第一台小型机 PDP-1，第一个系列计算机IBM 360

计算机功能部件=输入+输出+存储器+数据通路（运算器）+控制器

↑ (CPU=处理器=运算器+控制器)

(程序)



12 计算机的指令系统

指令系统

冯诺依曼结构计算机：**存储程序**计算机（程序-指令-存储器），**顺序执行指令**（PC-存储器-指令）

- 存储程序概念 stored-program concept：指令和数据都以二进制数形式存储在存储器。
- 指令和数据存在同一存储区域→冯诺依曼结构；分开存储→哈佛结构。
- （一些小问题：“存储程序”这个名词有时候被用作冯诺依曼结构的代称）

计算机系统 = 硬件 + 软件

- 软件：程序（由**指令**序列组成）

计算机语言=机器语言=01串 ← 汇编语言=助记符 ← 高级程序语言

(汇编器)

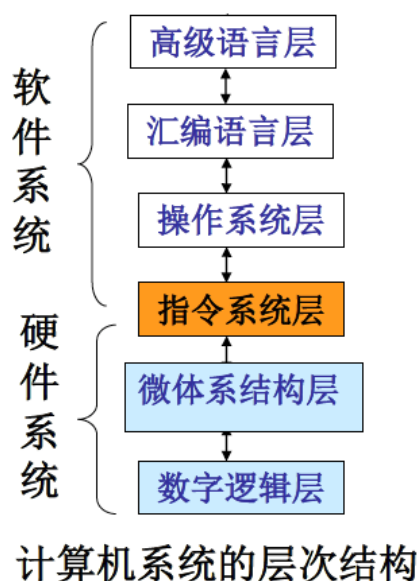
(编译器)

指令：计算机运行的最小功能单元，指挥计算机硬件运行的命令。一台计算机提供的全部指令构成它的**指令系统**。

- 硬件：中央处理器、存储器、外围设备等，功能是输入&输出&处理&存储数据

指令系统：硬件系统和软件系统之间，是硬、软件之间的接口部分。

指令系统分类：复杂指令集CISC，精简指令集RISC，超长指令字（例如Intel 安腾指令集）



指令

指令功能分类：数据运算指令、数据传输指令、控制指令、输入输出指令、其他指令

指令格式 instruction format：操作码、操作数地址的二进制分配方案

- 操作数地址字段可能需要指出①运算器中累加器的编号或专用寄存器名称、②输入输出指令用到的 I/O 设备的入出端口地址、③内存储器中的一个存储单元（或一 I/O 设备）的地址

【概念】

- 指令字：一条指令的二进制表示
- 指令字长：指令字中二进制的位数
 - 【区分】机器字长：计算机能够直接处理的二进制数据的位数（32-bit / 64-bit）；指令字长可以是0.5 / 1 / 2, ... 个机器字长
 - 根据指令字是定长还是变长分为 定长 / 变长指令字结构

计算机性能衡量指标

吞吐率、响应时间、MIPS、CPI、CPU Time、CPU Clock、综合测试程序

【注意】不一定CPI越小性能越好，需要综合考虑主频、不同指令集对同一程序产生的指令条数等因素。

13 数据表示及检错纠错

数据的编码与表示

字符型数据是重要的人机界面，字符集编码标准：ASCII，UNICODE，UTF-8

1. ASCII，占用**一个字节**（对齐到byte？），**7 位**二进制编码表示 128 个西文字符
2. UNICODE，**16 位**表示 65536 个字符，编码空间划分为块（16的整数倍），但依然无法覆盖所有字符
3. UTF-8 编码，**变长**，首字节为长度；除首字节均以 10 开始，可**自同步**；可扩展性强

4. 点阵字体：本质是单色位图（放大后不清晰），文字编码->查找字体文件->找到点阵->显示
5. 矢量字体：用曲线表示字体，每条曲线保存关键点（形状平滑）

逻辑型数据：0和1

数值型数据：定点数（有整数和定点小数，但后者很少用）、浮点数

- 整数的表示：MSB均为符号位，其余数值位
 - 原码：数的绝对值
 - 反码：正数不变，负数每位的绝对值取反
 - 补码：正数不变，负数每位的绝对值取反+1
- 【facts】
 - 0的原码和反码都有2种（符号位的0/1，+0/-0），但补码只有一个
 - 补码比原、反码多表示1个数，例如32位int，后二者是 $[-INT\ MAX, INT\ MAX]$ ，补码是 $[INT\ MIN, INT\ MAX]$ ，其中 $INT\ MIN = -INT\ MAX - 1$
 - 不能说负数的原码、反码、补码均不同，例如 -2^{30} ，原码和补码都是1100,0000,...,0000.
 - 浮点数加法不满足结合律
- 浮点数的表示： $(-1)^s M \cdot 2^E$ ，其中 $Bias = 2^{k-1} - 1$ （k为exp域的宽度）。
 - 规格化浮点数 $E = Exp - Bias$ ， $M = 1.xyzw$ ；因此 $Exp = 01111..111 = Bias$ 时 $E=0$ ， $2^E=1$ ；若frac域全0，则表示的数字为1。
 - 非规格化（Exp全0） $E = 1 - Bias$ 。

Dynamic Range (Positive Only)					$v = (-1)^s M 2^E$ $n: E = Exp - Bias$ $d: E = 1 - Bias$
	s	exp	frac	E	
Denormalized numbers	0	0000	000	-6	0
	0	0000	001	-6	$1/8 * 1/64 = 1/512$
	0	0000	010	-6	$2/8 * 1/64 = 2/512$
	...				
	0	0000	110	-6	$6/8 * 1/64 = 6/512$
	0	0000	111	-6	$7/8 * 1/64 = 7/512$
	0	0001	000	-6	$8/8 * 1/64 = 8/512$
	0	0001	001	-6	$9/8 * 1/64 = 9/512$
	...				
	0	0110	110	-1	$14/8 * 1/2 = 14/16$
Normalized numbers	0	0110	111	-1	$15/8 * 1/2 = 15/16$
	0	0111	000	0	$8/8 * 1 = 1$
	0	0111	001	0	$9/8 * 1 = 9/8$
	0	0111	010	0	$10/8 * 1 = 10/8$
	...				
	0	1110	110	7	$14/8 * 128 = 224$
	0	1110	111	7	$15/8 * 128 = 240$
	0	1111	000	n/a	inf

检错纠错码

码距：增大码距能提高检错能力。

- 循环冗余校验CRC（串行数据传输）
- 奇偶校验（并行）：k位数据码 + 1位校验码，码距变为2。
- 海明校验（并行）：k位数据码 + r位校验码，注意到数据码D的位置 ≥ 3 ，因此至少被2个校验码覆盖。
 - 纠一位错，则共有 k+r 种错，需要能够被r位校验码的错误组合覆盖（r位校验码有 2^r 种组合，其中1种 $\checkmark$ ，剩下 $2^r - 1$ 种错），即 $k + r \leq 2^r - 1$ ，此时码距为3（需要1处数据错+至少2个校验位错）
 - 纠一位错&查两位错，用于海明校验的有r-1位，最后1位为奇偶校验，将上式的r换成r-1即可，此时码距加上奇偶校验为4

- 下列通用算法可以为任意位数字产生一个可以纠错一位（英语：Single Error Correcting）的汉明码。
 1. 从1开始给数字的数据位（从左向右）标上序号, 1, 2, 3, 4, 5...
 2. 将这些数据位的位置序号转换为二进制, 1, 10, 11, 100, 101,等。
 3. 数据位的位置序号中所有为2的幂次方的位（编号1, 2, 4, 8, 等，即数据位位置序号的二进制表示中只有一个1）是**校验位**。
 4. 所有**其它位置**的数据位（数据位位置序号的二进制表示中至少2个是1）是**数据位**。
 5. 所有校验位覆盖了**数据位置**和**该校验位位置的二进制表示AND得到的值不为0**的数，如校验位1覆盖了所有数据位位置序号的二进制表示倒数第一位是1的数据，2 - 倒数第二位，4 - 倒数第三位，8 - 倒数第四位。
- **例子** [\[编辑 \]](#)

对11000010进行汉明编码，求编码后的码字。

- 1.列出表格，从左往右（或从右往左）填入数字，但2的次方的位置不填。

位置	1	2	3	4	5	6	7	8	9	10	11	12
数据			1		1	0	0		0	0	1	0

- 2.把**数据**行有1的列的**位置**写为二进制。

位置	1	2	3	4	5	6	7	8	9	10	11	12
数据			1		1	0	0		0	0	1	0
二进制			0011		0101						1011	

- 3.收集所有二进制数字，求**异或**。 $0011 \oplus 0101 \oplus 1011 = 1101$
- 4.把1101依次填入表格中2的次方的位置（低位在左）。

位置	1	2	3	4	5	6	7	8	9	10	11	12
数据			1		1	0	0		0	0	1	0
二进制			0011		0101						1011	
校验	1	0		1				1				

- 5.所以编码后的码字是101110010010。

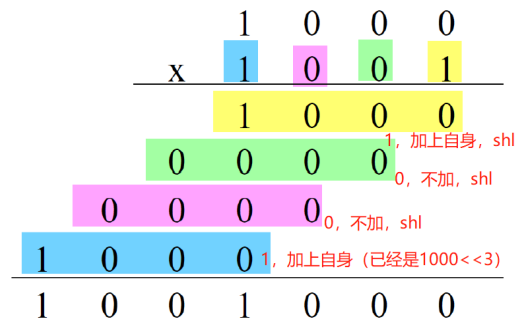
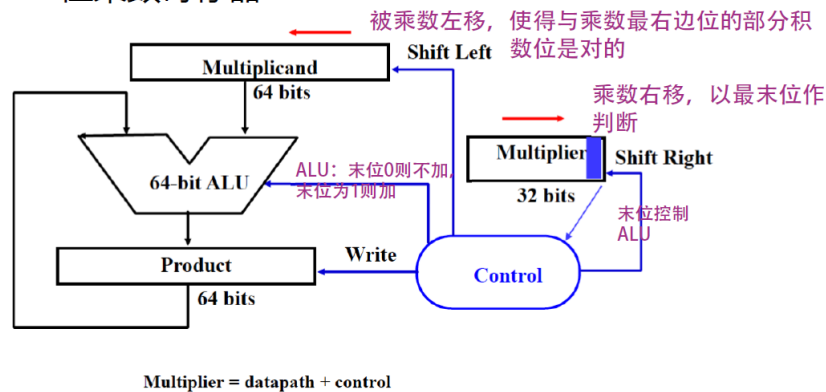
14 算术运算及其电路实现

二进制乘法的原码乘法（一）：

被乘数寄存器用64-bit存32-bit，浪费空间；ALU每次64-bit加法只有一半位有效，浪费计算能力。

原码乘法的实现（一）

- ▶ 64-位被乘数寄存器，64-位ALU，64-位部分积寄存器，32-位乘数寄存器

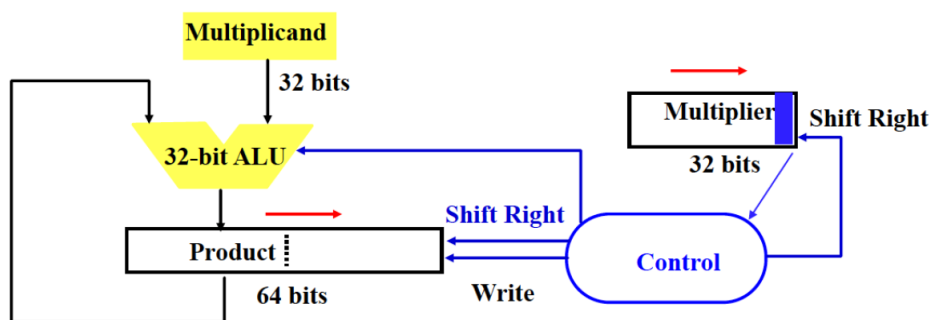


原码乘法（二）

（一）的基础上改进，乘积一定是64位，所以32-bit被乘数和32-bit ALU，每次取乘积的高32-bit，完了之后乘积右移。

原码的乘法实现（二）

- ▶ 32位被乘数寄存器，32位ALU，64位部分积寄存器

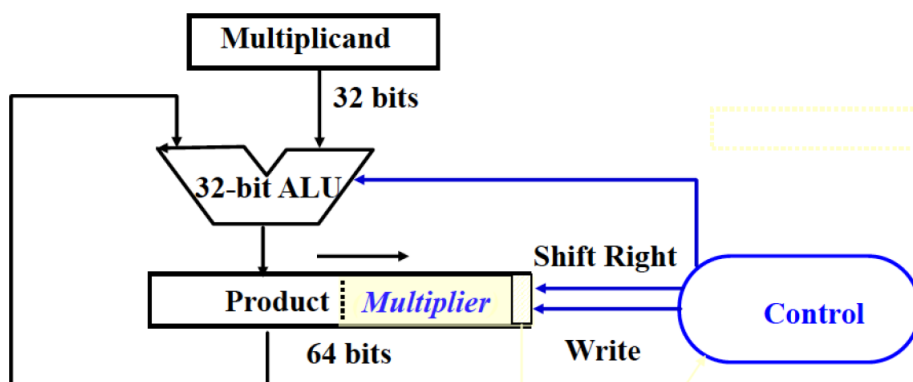


原码乘法（三）

节省乘数寄存器，直接放在乘积的低32位，因为每步之后它们都会右移，同时右移不会打架！

原码乘法实现（三）

- ▶ 32位被乘数寄存器，32位ALU，64位部分积寄存器（0-位乘数寄存器）



补码直接做乘法：布斯算法

推导：

补码乘法运算

$$\oplus [x*y]_{\text{补}} = [x]_{\text{补}} * (-2^{n-1}y_{n-1} + \sum_{i=0}^{n-2} y_i 2^i) \mod 2^n$$

$$\oplus = [x]_{\text{补}} * [2^{n-1}(y_{n-2} - y_{n-1}) + 2^{n-2}(y_{n-3} - y_{n-2}) + \dots + 2^0(y_{-1} - y_0)]$$

写成相邻项差的形式（因为 $2^{n-1}=2*2^{n-2}$ ，作差还是一样）

$$\oplus = [x]_{\text{补}} * \sum_{i=0}^{n-1} 2^i (y_{i-1} - y_i) \quad \text{其中：} y_{-1}=0$$

$[y]_{\text{补}}=y_{n-1}y_{n-2}\dots y_0$

流程：

- $[x]_{\text{补}}=X$, $[y]_{\text{补}}=Y$, 附加位初始化 $y_{-1}=0$, 部分积用两倍宽度（即 $2n$ ）的寄存器，低半边存 Y
- 在第 i 步，检查 Y 的最后一位 y_i 和上一被移出的 y_{i-1} （存在附加位），计算 $y_{i-1} - y_i$ ，为1则部分积 $+=X$ ，0则不变，-1则部分积 $-=X$ 【注意是附加位-末位！】
- 部分积右移，移出 y_i 到附加位， $i=n-1$ 时终止，小于时进行第 $i+1$ 步

原码除法：加减交替算法

注意这里的 $-Y$ 和 $+Y$ 是通过 $+[-Y]_{\text{补}}$ 和 $+ [Y]_{\text{补}}$ 完成的。部分积和商每步之后都需要左移1位，除了最后一步

被除数(余数)	商	
001011	00000	开始情形
+ 110011		-Y
111110	00000	<0, 商0
111100	00000	左移1位
+ 001101		+Y
001001	00001	>0, 商1
010010	00001	左移1位
+ 110011		-Y
000101	00011	>0, 商1
001010	00011	左移1位
+ 110011		-Y
111101	00011	<0, 商0
111010	00011	左移1位
+ 001101		+Y
000111	00101	>0, 商1

21 控制器概述、指令和指令系统

控制器的作用：根据指令要求提供给各部件相应控制信号，指挥协调各部件共同完成指令，自动执行下一条指令

指令格式：操作码 | 操作数地址，其中操作码的组织与编码

- 定长：可以简化硬件设计，提高译码和识别速度；
- 变长：可以表示更多的指令，但增加了译码和分析难度；
- 操作码和操作数地址字段交叉：特殊，少用
- 【扩展技术】根据指令操作数个数不同，使用变长操作码（基础+扩展）实现。

指令操作码扩展技术

- 按顺序!!
- 前面介绍的操作码扩展方案中，每次扩展4位并仅保留了一个编码用于接下来的扩展过程，当每次扩展的位数和保留的位数变化时，后面可扩展的指令条数就可以变化。例如在16位字中的指令字中，可以选用如下方案支持三地址指令、二地址指令、一地址指令和零地址指令14、30、31、16条：
 - ▶ 14条三地址为：0000 ~ 1101 (保留1110, 1111 两个码)
 - ▶ 30条二地址为：11100000 ~ 11111101 (保留2个码) 0000 ~ 1101
 - ▶ 31条一地址为：11111100000 ~ 11111111110 (保留1个码) 只留
 - ▶ 16条零地址为：11111111110000 ~ 11111111111111 全

22 RISC-V指令系统

lw / sw需要四字节对齐，lb / sb不用；因此实现一个字节的传输可以用lw + 掩码，也可以用lb。

字节排布：大端 vs 小端

- ▶ 大端机：最高的字节在最低的地址，字的地址等于最高字节的地址
- ▶ 小端机：最低的字节在最低的地址，字的地址等于最低字节的地址

高地址 * (\$s0) = 0x00000180 低地址

80	01	00	00
----	----	----	----

Big Endian

00	00	01	80
----	----	----	----

Little Endian

RISC-V 是小端机

字节数据传输指令：sb指令的高24位被忽略，lb指令的高24位做符号扩展；半字lh也同理（高16位）；

算术指令的addi：imm被符号扩展

23 指令格式与数据通路设计

指令格式

7位				5位				5位				3位				5位				←小端机															
31	30	25	24	21	20	19	15	14	12	11	8	7	6	0																					
funct7				rs2				reg source				rs1				funct3				rd				dest				opcode				R-type			
imm[11:0]												rs1				funct3				rd				opcode				I-type							
imm[11:5]						rs2				rs1				funct3				imm[4:0]				opcode				S-type									
imm[12]		imm[10:5]				rs2				rs1				funct3				imm[4:1]		imm[11]		opcode				B-type									
imm[31:12]												对于大imm, 20位储存												rd				opcode				U-type			
imm[20]		imm[10:1]				imm[11]				imm[19:12]				rd				opcode				J-type													

【注意】

- I 类型包括jalr，即最高位是imm[11:0]，jalr rd, rs, imm 返回地址存在rd里，r意为跳转地址是 PC<=Reg[rs]+imm。所有imm都会从12位符号扩展为32位，所以和之后lui结合addi的情况一样要注意sign extension trick（比如先给lui的高20位加上1）

Call function at any 32-bit absolute address

lui x1, <hi20bits>

jalr ra, x1, <lo12bits>

Jump PC-relative with 32-bit offset

auipc x1, <hi20bits> # Adds upper immediate value to
and places result in x1

jalr x0, x1, <lo12bits> # Same sign extension trick needed
as LUI

- *J* 类型只有一条jal, jal rd, imm 是将返回地址存在rd, 跳转地址为 $PC \leftarrow PC + imm$ 。其中立即数最高位是imm[20]在第32位, 然后排布是[20, 10:1, 11, 19:12], 为了和U指令对齐; 另外, 由于跳转地址是2的倍数, 可以只表示20:1位, imm最后一位一定是0 (类比B指令)
- *B* 类型填了imm[12:1], 末位为0
- *U* 类型是imm[31:12], 包含了 lui 和 auipc, 装入20位立即数到高20位 (5位hex), 低12位清零。lui和addi结合使用时要注意 *I* 类型的符号扩展 (如下图), 0xE=1110(binary), addi会符号扩展成0xFFFF FEEF, 加上0xDEAD B000相当于0xDEAD BEEF - 0x0001 0000, 就变成了0xDEAD AEEF, 解决办法是lui之前先给高20位+1

使用lui的例子

1位16进制=4位2进制
3×4=12

▶ lui设置了高20位, 通过addi设置低12位

▶ 例1: 设置0x87654321

```
lui    x10, 0x87654      # x10=0x87654000
addi   x10, x10, 0x321    # x10=0x87654321
```

✱ 例2: 设置0xDEADBEEF

```
lui    x10, 0xDEADB      # x10=0xDEADB000
addi   x10, x10, 0xEEF    # x10=0xDEADAEEF
```

解决办法: lui 高20位 加1 认为要符号扩展 (相当于高位
伪指令li x10, 0xDEADBEEF: 自动生成两条指令 加了11111
也就是+(-1))
所以少了1.

▶ 32

修正
(16也要考虑)

数据通路设计

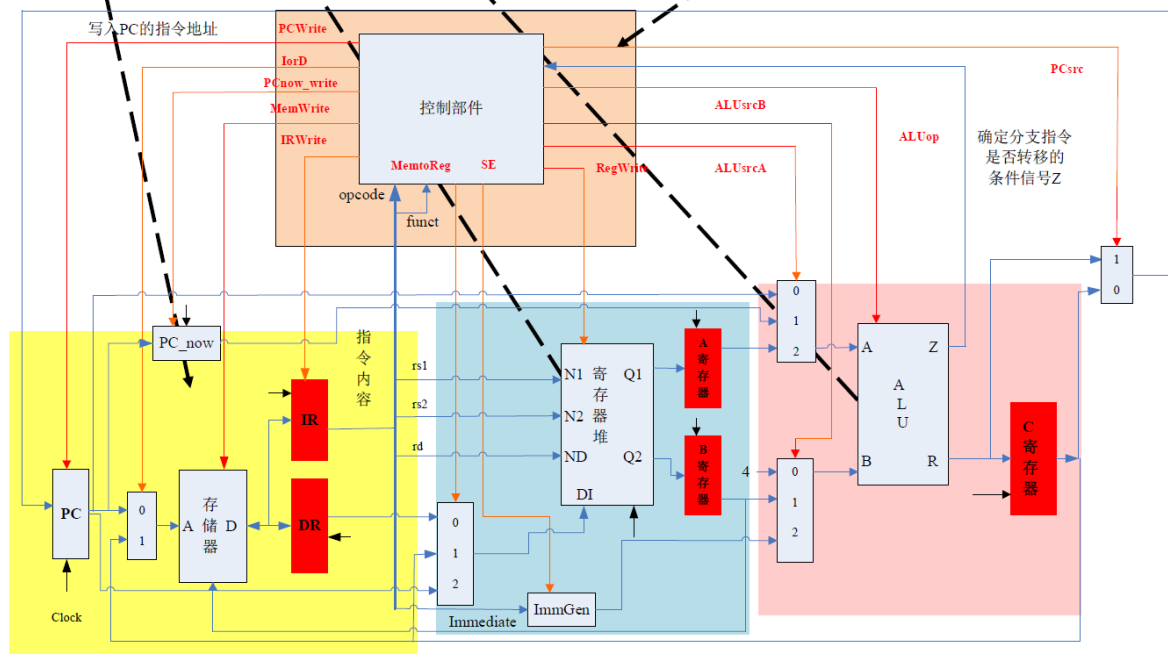
单周期CPU: CPI=1

多周期CPU: CPI≈4, 因为大多数指令没有MEM

流水线CPU: 尖峰CPI=1 (?)

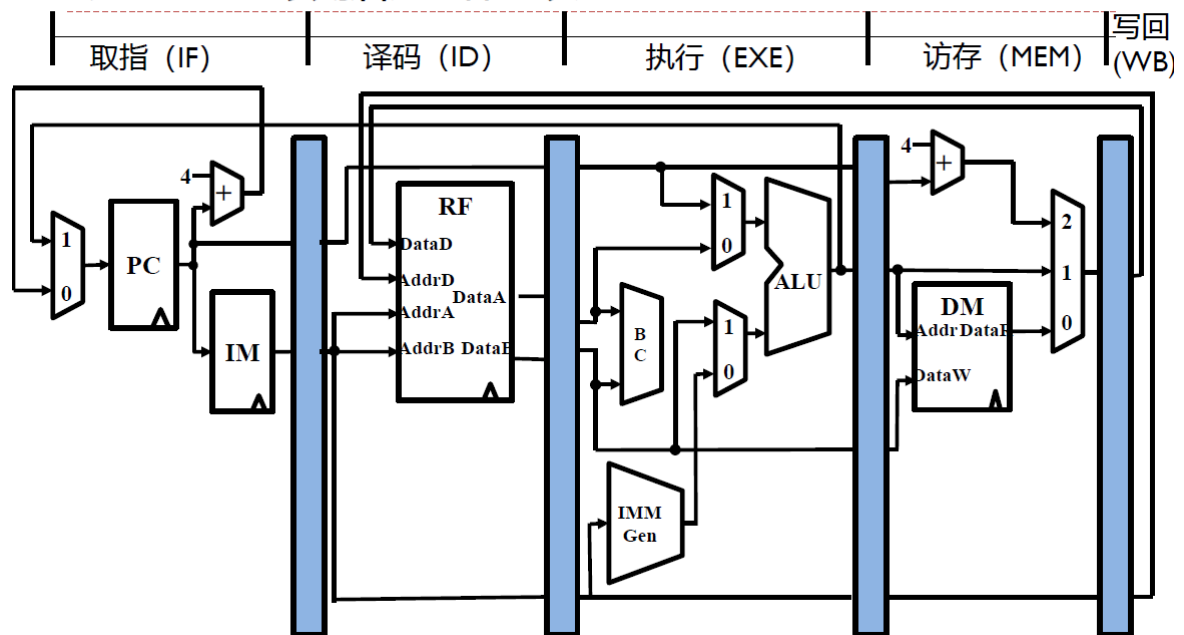
24 单周期CPU

由存储器、寄存器堆、ALU部件、控制部件 4部分组成

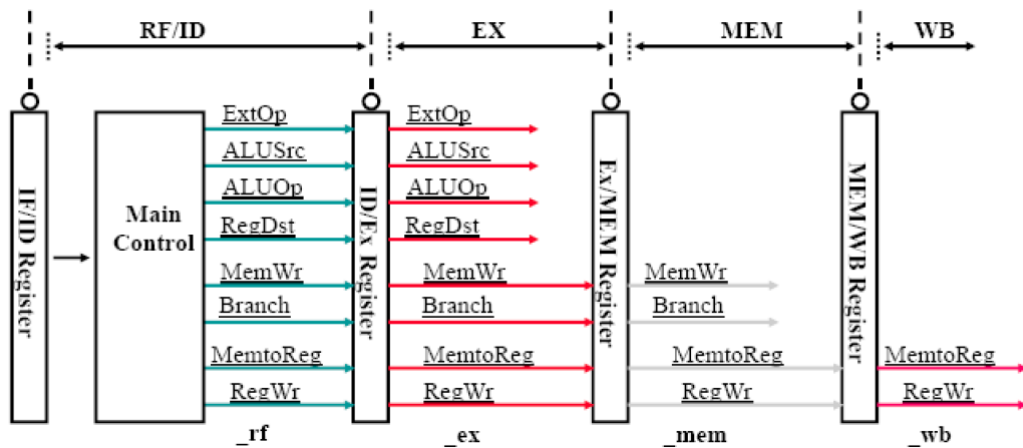


26 流水线技术概述

流水线的数据通路划分



流水线控制的实现



流水线的表示方法

- 连接图：？
- 时空图：有两种画法，横轴都是时间 t ，纵轴可以是指令执行阶段，也可以是程序执行顺序（指令序列）

流水线寄存器：保存了上一阶段传给下一阶段的所有数据、控制信号

流水线分类：部件功能级流水线（运算操作流水线）、处理机级流水线（指令流水线）、处理及间级流水线（宏流水线）

衡量流水线性能的指标：吞吐率（单位时间执行指令数），加速比（与串行相比速度提高的比率）

27 流水线结构冲突&数据冲突

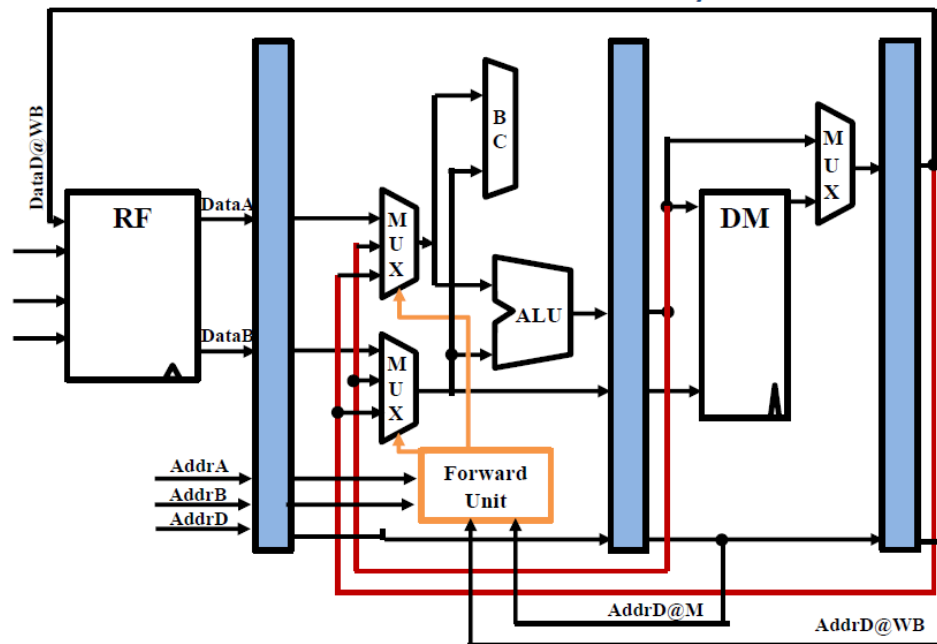
结构冲突：【总结：存储器争用冲突，同一存储器可以双端口访问，或者使用不同存储器以解决】

- 寄存器结构冲突 - 同时读写？
 - 可以用不同的端口解决，两个read ports，一个write port，彼此独立【注意在实现中特判了 $raddr == waddr$ 就给 $wdata_i$ 的情况，因此实现了WB的data forwarding】
 - 双沿访问，下降沿写入，上升沿读出 —— 可惜FPGA做不到
- 内存结构冲突 - IF&MEM同时访存
 - stall，进行仲裁，总线给MEM，IF被插入气泡进入等待
 - 分离IM、DM，或者至少分离Inst Cache、Data Cache → 哈佛结构

数据冲突：（属于局部冲突）

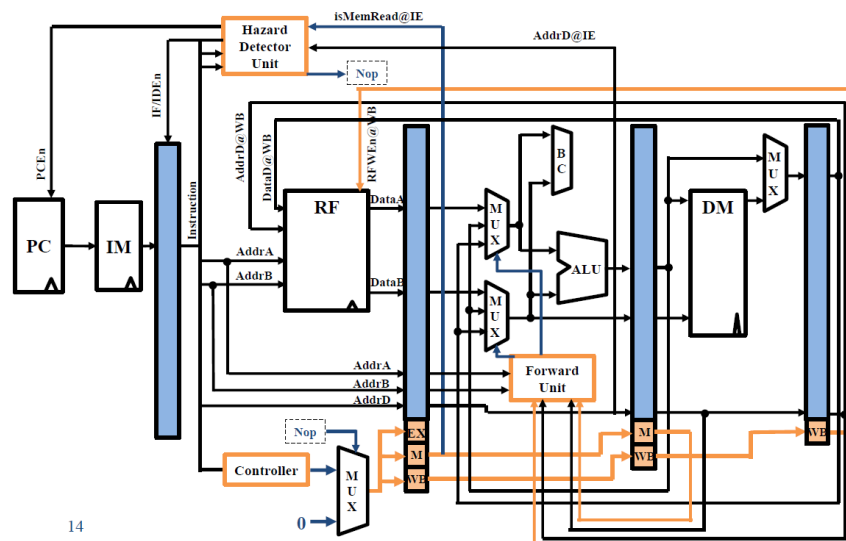
- 写后读冲突（RAW）：WB才会写回，但可能下一/两条指令的ID就需要使用了（此时上一条指令至多到EX(对下一条ID) / MEM(对下两条ID)）
 - R类型 - 数据前推（旁路，data forwarding）：课件上是EX/MEM、MEM/WB流水寄存器的输出数据前传到EX阶段中ALU输入端口；造机实现的时候是EX、MEM的输出数据（即EX/MEM、MEM/WB的输入数据）前传到ID（冲突解决的位置整体往前移动一拍）。【注意】要特判不是0寄存器 x_0 !

➔ Pictured: ALU → ALU in one and 2 cycles



◦ load/store类型

- 插入一个空拍，然后可以获得前推的数据，或者
- 汇编器静态调度，汇编时调整指令顺序，避免下一条装入依赖于lw rd, imm(rs)中rd的指令
- 硬件（处理器，即CPU）动态调度，指令乱序流出，会产生异常处理不够精确（异常指令后面的先执行完了，然后才检测到异常.....）



- 写后写冲突（WAW）：RISC-V不会发生这种冲突，因为流水线暂停会把之后的全部暂停，而且只有WB会写回，因此一定是严格按指令序列进行写的
- 读后写冲突（WAR）：RISC-V不会发生这种冲突，因为ID完成所有的寄存器读，WB完成所有的寄存器写，因此在后面的指令的写操作一定发生在前面的指令的读操作之后

28 控制冲突和异常

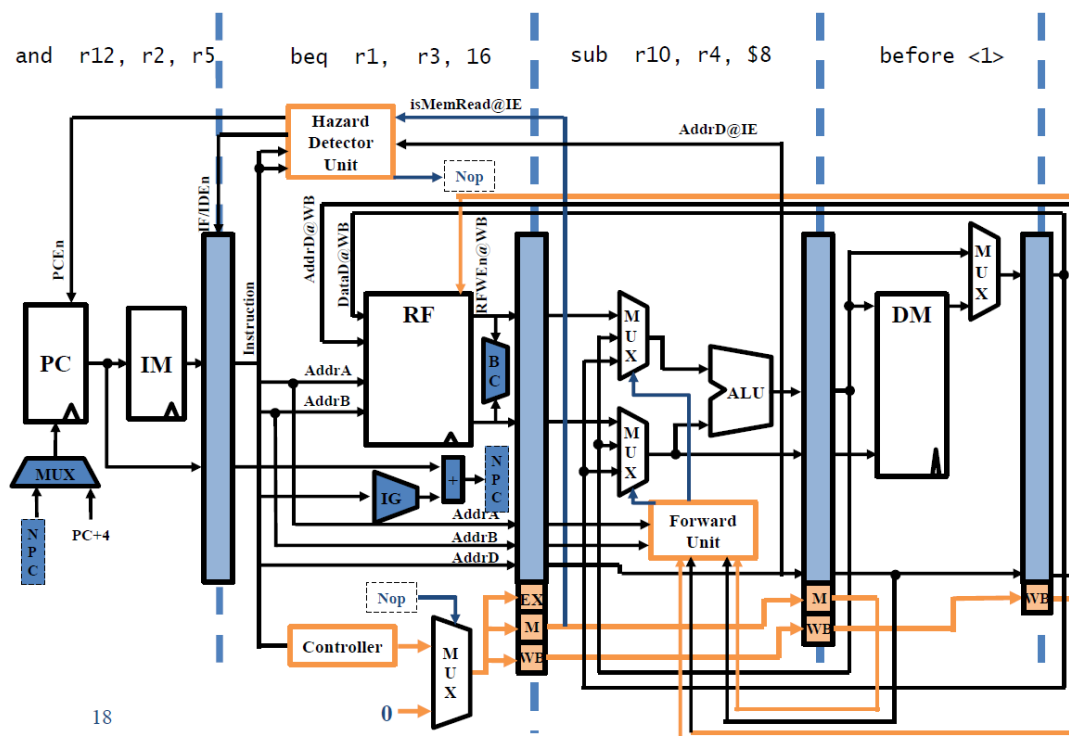
控制冲突：（属于全局冲突）分支指令

- EX阶段（stall住后面的）生成转移到newPC，1个周期传入PC，1个周期传入IF；以分支指令的IF为1，那么在3（EX）+2（传两个周期）=5时可以正常流动，**暂停了两拍**

指令	时钟									
	1	2	3	4	5	6	7	8	9	10
分支指令	IF	ID	EX	MEM	WB					
分支后继指令		IF	Stall	Stall	【IF】	ID	EX	MEM	WB	
分支后继指令+1			EXE算出newPC,			IF	ID	EX	MEM	WB
分支后继指令+2			1个周期传到PC输入端				IF	ID	EX	MEM
分支后继指令+3			1个周期从PC进IF					IF	ID	EX
分支后继指令+4									IF	ID
分支后继指令+5										IF

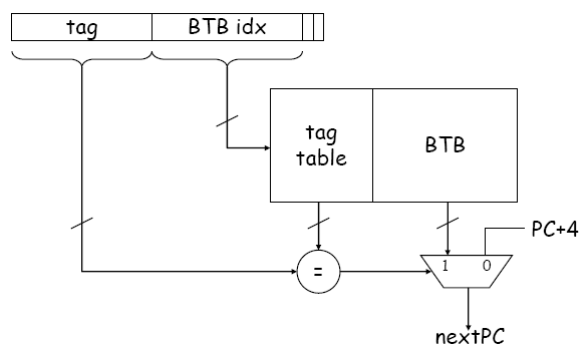
- 改进：提前到ID阶段就生成newPC，在2+2=4时可以正常流动，暂停了一拍
- 【增设部件】 $PC \leq MUX(NPC, PC+4)$

Branch Taken

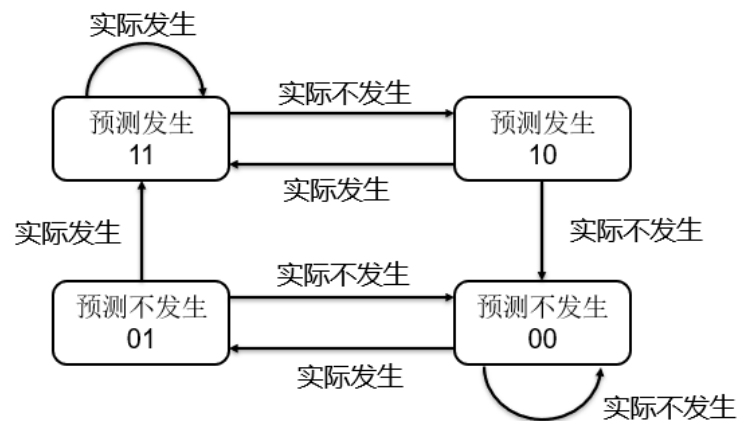
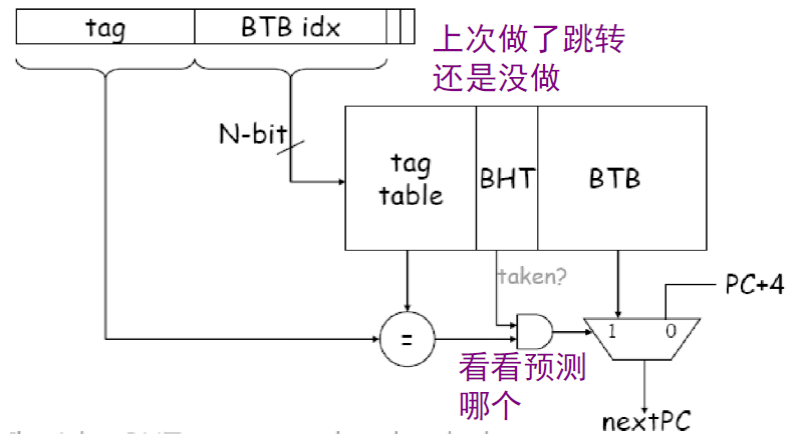


更大的改进办法：分支预测

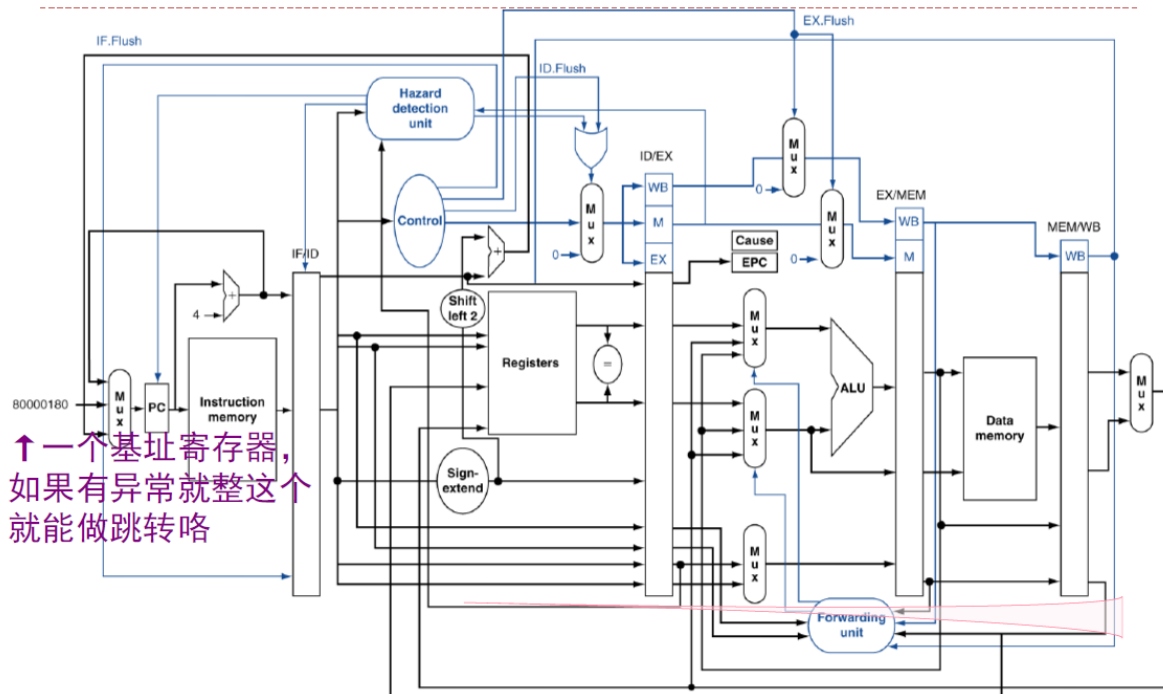
- 静态预测：可以用BTB，也可以不用
 - 一直预测不跳转，则在发现需要跳转的时候flush之前的指令，装入正确指令
 - 一直预测跳转（？有争议）
- 动态预测：使用BTB
 - BTB**: key=pc, value=next-pc; 为了减小大小，可以只保留key类型为 *B* 或 *J* 的 PC; 组织上可以组织成直接映射，BTB index放在低位（下图为静态预测）



- o **BHT**: 1位 / 2位 / ..., 看之前该条指令的跳转历史 (实现**动态预测**) , 如果是2位的, 要连续2次预测错误才改变预测方向。



流水线CPU的异常处理



31 DRAM

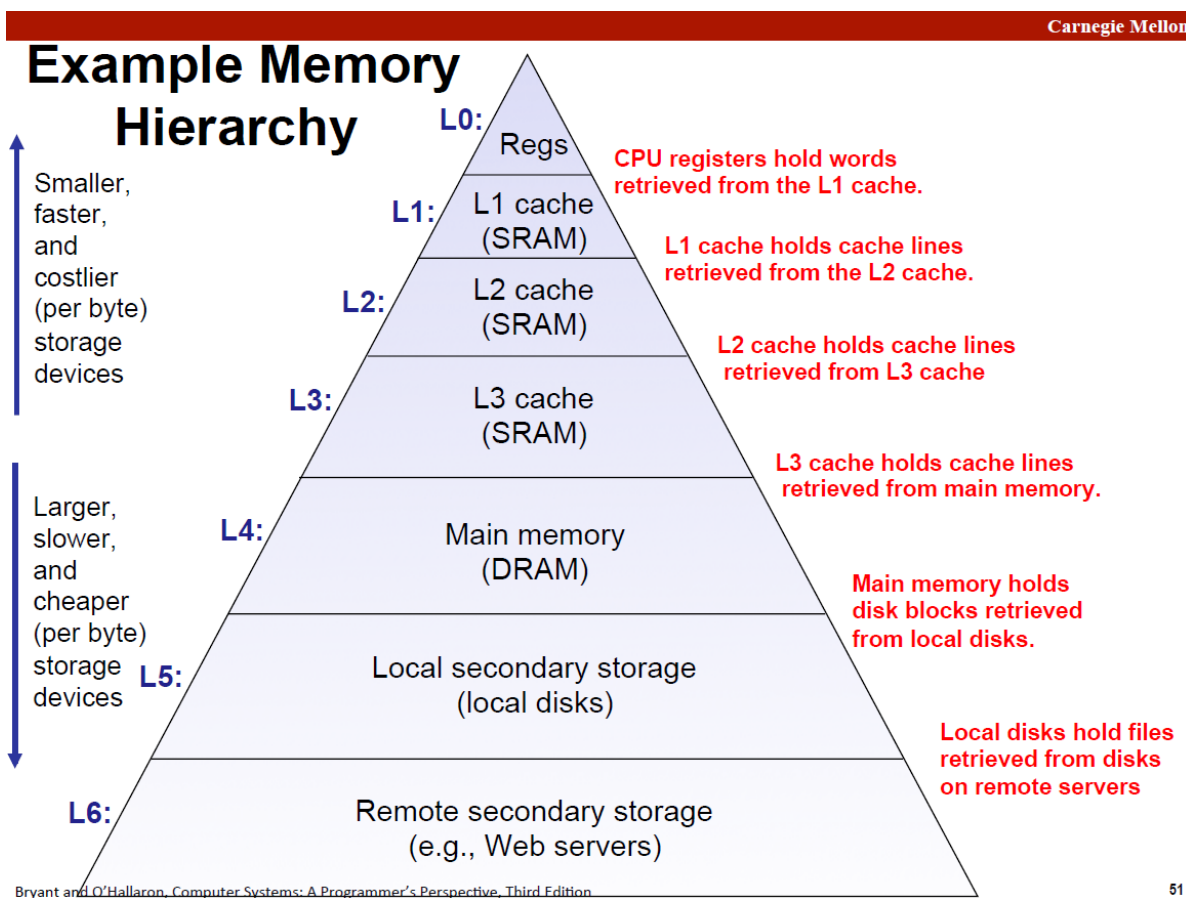
层次存储器系统

层次存储器系统设计目标: 高速度、大容量、低成本、高可靠性

层次之间的原则:

- 一致性原则（不同层次存储器如cache & 主存中同一信息要有同一个值）
- 包含性原则（内层的信息一定可以在外层被找到）

现代计算机存储器系统：主存储器（Register、Cache、Main Memory）、辅助存储器即外存储器（Magnetic disk、Tape...）



51

主存储器

作用：通过数据总线DB、地址总线AB、控制总线与CPU等其他部件连通

- 地址总线：用于选择主存储器的一个地址单元（存储大小可以是1 byte / word），位数width决定了【能够访问的存储单元的最大个数】，即【最大可寻址空间】。实际可以访问的存储空间大小还要结合按？寻址来确定（按字节 / 字寻址）
- 数据总线：位数width × 总线时钟频率 正比于 总线的最高数据吞吐能力
- 控制总线：指明总线的工作周期类型（如果是同步传输？）和本次入/出完成的时刻

动态存储器

存储原理：1个晶体管+1个电容

工作特点

- 破坏性读出（电容放完电就没了）
- 需定期刷新（电容会漏电，所以需要读出之后重新写回存储单元）
 - 集中刷新：停止读写，所有行逐行刷新
 - 分散刷新：每次完成读写之后刷新这一行
- 快速分页组织：连续的读写中用到了相同的行地址，就只送列地址，节省送地址的时间【DRAM送行列地址是分两次的】

33 静态存储器及高速缓冲存储器

静态存储器

存储原理：6个晶体管，触发器存储

SRAM与DRAM的区别：SRAM是触发器保存，因此不会破坏性读出也不要刷新，它的行列地址是同时送的

高速缓冲存储器 (Cache)

主存和CPU之间的存储器，用SRAM实现。

特点：高速、透明（程序员不可见）

作用：提高CPU的平均访存速度，原理是

- 时间局部性：最近被访问的信息很可能很快还要被访问，所以装入cache
- 空间局部性：最近被访问的信息邻近的信息很可能要被访问，所以访问过的信息临近的信息也一起装入cache

要解决的问题：地址和Cache行的映射关系、数据之间一致性的保持、数据交换的粒度（块=行）、Cache内容装入和替换的策略

【与性能有关的概念】

- 命中 (hit)：命中率，命中时间（访问cache的时间）
- 缺失 (miss)：1-命中率，缺失时间（访问cache+主存的时间）
- 平均访问时间 = 命中率 × 命中时间 + (1-命中率) × 缺失时间

全相联方式

组数 (1 set) = 1，组内有很多路 (E way)，即很多个并行的块。

PC只被划分为标记位和块内地址，即tag | block offset

【计算】假设按字节寻址，则存储单元大小为1 byte

- 主存4GB => **地址存储单元个数为4G**，32位，则pc总位数是32
- Cache大小4KB，块大小4B => 块个数=4KB / 4B = 1K (10位)，又**set为0位**，则**E路的E=10**
- 块大小4B => **块内地址有4个，block offset占2位**，则tag位数=32-2=30

特点：

- 主存的字块可以和Cache的任何字块对应，利用率高，方式灵活（**不会出现conflict miss!**）
- **tag位比较长**，使用比较电路成本高

直接映射方式

组数需要计算，组内只有一路 (E=1) 即一组只有一个块。

PC被划分为标记位、组索引和块内地址，即tag | set index | block offset

【计算】假设按字节寻址，则存储单元大小为1 byte

- 主存4GB => 地址存储单元个数为4G，32位，则pc总位数是32
- Cache大小4KB，块大小4B => 块个数=4KB / 4B = 1K (10位)，又**组内只有一路**，则**组数=10**，即set index占10位
- 块大小4B => 块内地址有4个，block offset占2位，则tag位数=32-10-2=20

改进：增加块大小，可以更好地利用空间局部性

特点：

- 主存的字块只能和固定的Cache字块对应（取模，即取block offset对应的位），方式直接，利用率低（conflict miss难以避免）
- tag位比较短，比较电路成本低，而且每组只需要比较一次

多路组相联

PC被划分为标记位、组索引和块内地址，即tag | set index | block offset，与直接映射划分方式相同。

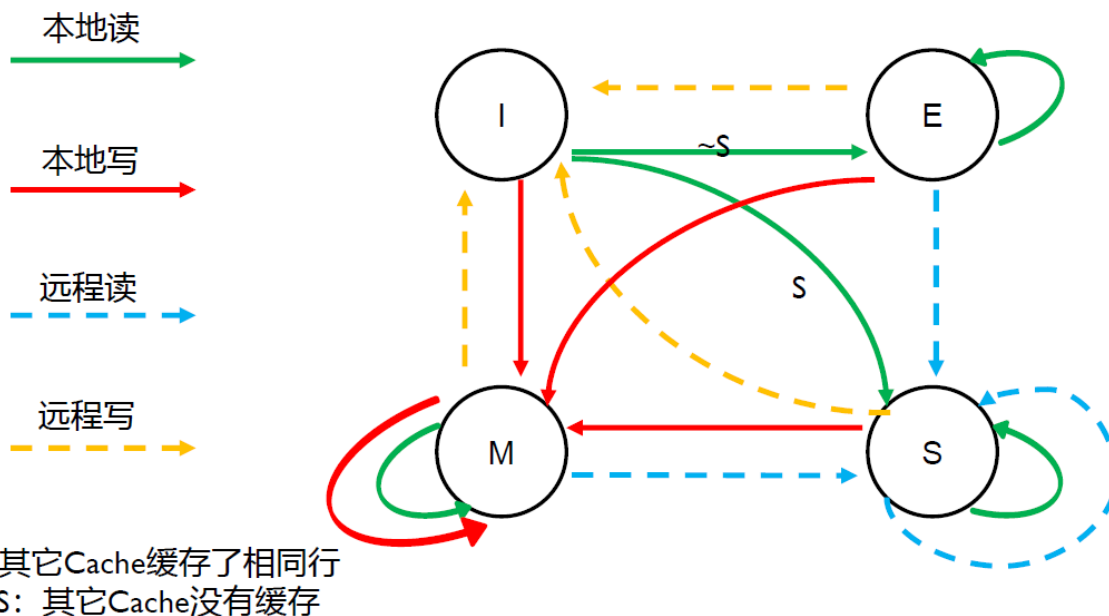
保持层次一致性的方法

解决写了Cache之后何时更新主存的办法：

- 写直达（write through）：强一致性
 - 串口一定要采用这种方式
 - 若Cache中命中了，两处都要改；若未命中，可以有写分配 / 非写分配两种方法
- 拖后写（write back）：弱一致性，替换cache时再写回主存
 - 通过监听总线访问实现
 - 很复杂，但效率高

一致性保证策略（MESI）：缓存行的状态转换（M修改态，E独占态，S共享态，I无效态）

缓存行的状态转换



Cache参数对性能的影响

根据公式，提高存储访问的性能的方式有：

- 提高Cache命中率，这和Cache设置参数有关，见下
- 其他：缩短缺失时的访问时间、提高Cache本身的速度

Cache缺失的原因（3C1I）：

1. 必然缺失 Compulsory Miss 【都会发生】

2. 容量缺失 Capacity Miss 【全相联经常发生】

- 方法：增大Cache容量

3. 冲突缺失 Conflict Miss 【直接映射经常发生】

- 方法：增大Cache容量 / 相联的组数

4. 无效缺失

其他减小缺失率的方法①：一定程度地增大块大小可以利用好空间局部性（缺失率↓），但增大到过大（如果Cache容量一定）则块的个数减少（缺失率↑），而且block增大必然导致缺失损失增大（装入数据块的时间↑）

其他减小缺失率的方法②：多级Cache（相当于增大了Cache？）；或者分为指令Cache和数据Cache

块替换策略

直接映射：一个块映射到固定组的固定位置（组里就一个位置），故不需要选择，直接替换。

全相联：一个块可以映射到组（就一个组）的任意位置，需要选择。

N路组相联：一个块可以映射到一个组里的N个位置，需要选择。

替换策略：LRU、FIFO、RAND

Cache接入系统的体系结构

1、侧接法：接到总线上（总线占用率高）

2、隔断法：CPU - Cache ----总线---- 其他I/O设备，有利于提高总线利用效率，缺点是结构复杂成本高

34 虚拟存储器

虚存：由操作系统管理，进行**主存和外存数据交换**，功能有

- 它将**主存**看成是一个存储在**磁盘**上的地址空间的高速缓存，在主存中只保存活动区域，并根据需要在磁盘和主存之间来回传送数据，通过这种方式，它高效地使用了主存。
- 它为每个进程提供了一致的地址空间，从而简化了内存管理。它还能实现进程间的内存共享。
- 它保护了每个进程的地址空间不被其他进程破坏（页表中存放了访问权限）。

【原理】程序的局部性，但是如果活动区域的集合大小 > 主存容量，那么会出现抖动（thrashing）现象，活动页不断被装入。

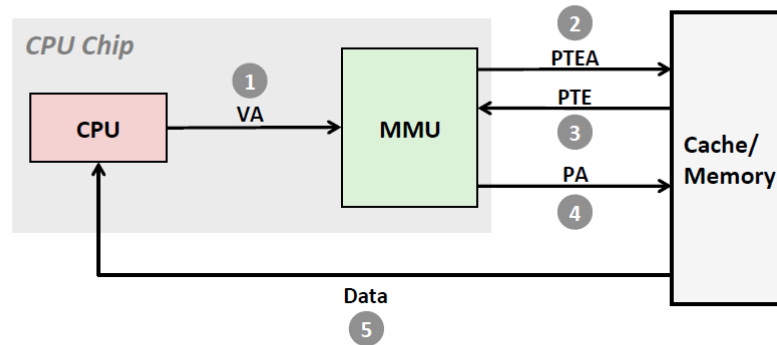
页表

页式存储管理：page size固定（以byte为单位），将主存、虚存按页划分，通过页表进行管理。

页面大小：减小了可以减少内部碎片（没有用到的部分？），但是需要增大页表，所以趋势是增大页面大小。

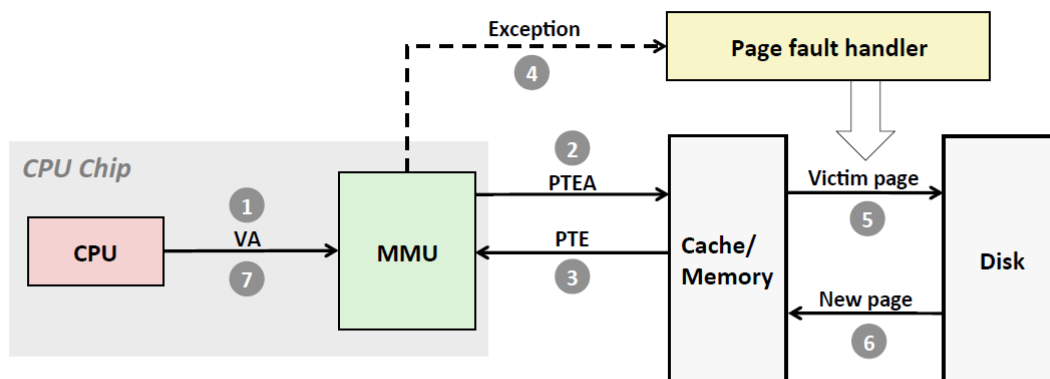
页面替换算法：OS读页表（一般是TLB miss后）发现需要的页未装入主存，则指示CPU访问外存，从外存调入页到主存，若主存已满则进行页面替换，修改完了返回。

Address Translation: Page Hit



- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) MMU sends physical address to cache/memory
- 5) Cache/memory sends data word to processor

Address Translation: Page Fault



- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) Valid bit is zero, so MMU triggers page fault exception
- 5) Handler identifies victim (and, if dirty, pages it out to disk)
- 6) Handler pages in new page and updates PTE in memory
- 7) Handler returns to original process, restarting faulting instruction

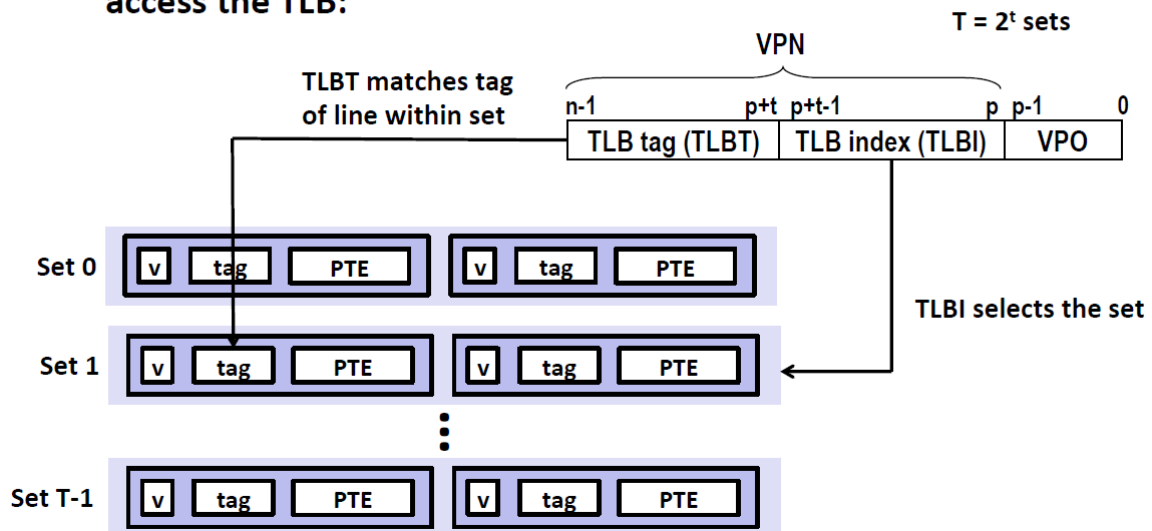
TLB

TLB缺失将导致：流水线停止，通知OS，OS读页表，将页表项写回TLB，返回用户态，重新访问。

减少缺失的方法：多路组相连、提高TLB容量。

Accessing the TLB

- MMU uses the VPN portion of the virtual address to access the TLB:



- TLB hit: 则不需要访问内存中的页表
- TLB miss: 则需要访问内存中的页表

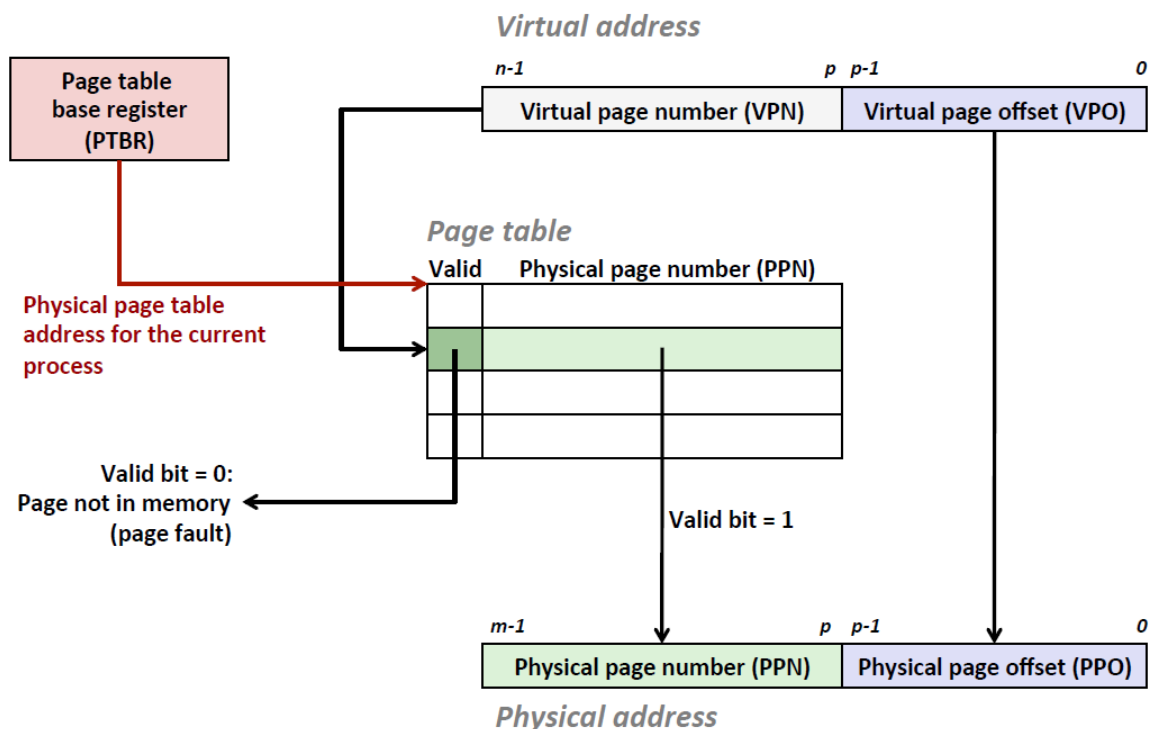
地址翻译

$P = \text{page size} = 2^p$, 则 $VPO = PPO$ 为 p 位 ($0 \sim p-1$)

设 $N = \text{VA 总数} = 2^n$, 即 VA 有 n 位, 则 VPN 为 $n-p$ 位; 设 $M = \text{PA 总数} = 2^m$, 即 PA 有 m 位, 则 PPN 位 $m-p$ 位

页表实现把 VPN $\rightarrow$ PPN:

Address Translation With a Page Table



Review of Symbols

■ Basic Parameters

- $N = 2^n$: Number of addresses in virtual address space
- $M = 2^m$: Number of addresses in physical address space
- $P = 2^p$: Page size (bytes)

■ Components of the virtual address (VA)

- TLBI: TLB index
- TLBT: TLB tag
- VPO: Virtual page offset
- VPN: Virtual page number

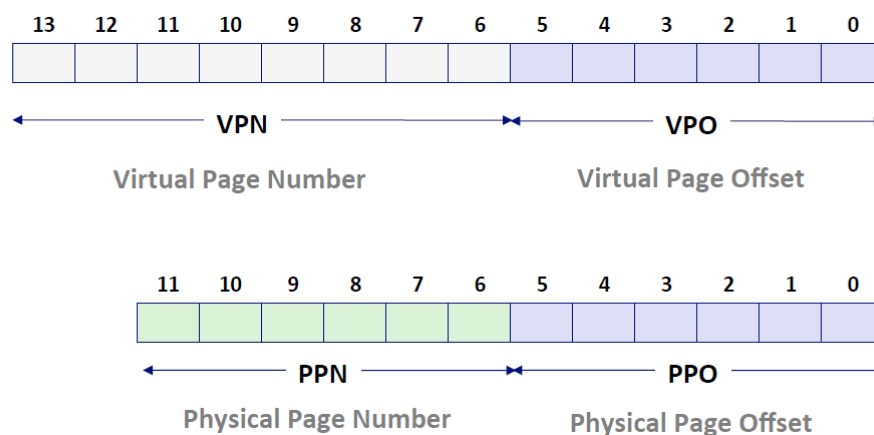
■ Components of the physical address (PA)

- PPO: Physical page offset (same as VPO)
- PPN: Physical page number
- CO: Byte offset within cache line
- CI: Cache index
- CT: Cache tag

Simple Memory System Example

■ Addressing

- 14-bit virtual addresses
- 12-bit physical address
- Page size = 64 bytes

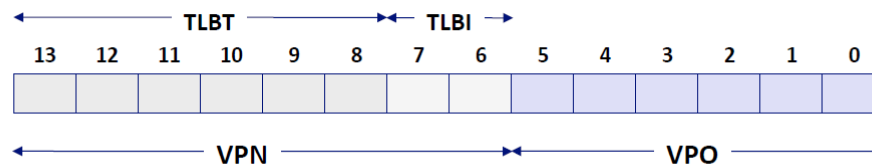


给出VA为14位, PA为12位, page size=64bytes, 则: VPO和PPO为 $\log_2 64 = 6$ 位, VPN=14-6=8位, PPN=12-6=6位

TLB一共16个表项, 组织为4路组相联, 则: 组数set number=16/4=4, 索引TLBI位数= $\log_2 16/4 = 2$, 标志TLBT位数=VPN位数 - TLBI位数 = 8-2 = 6 (图中是2位16进制数, 实际可表示8位, 计算时应当截断为6位, 即0x0A=001010₍₂₎)

1. Simple Memory System TLB

- 16 entries
- 4-way associative



Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	0
2	02	–	0	08	–	0	06	–	0	03	–	0
3	07	–	0	03	0D	1	0A	34	1	02	–	0

2. Simple Memory System Page Table

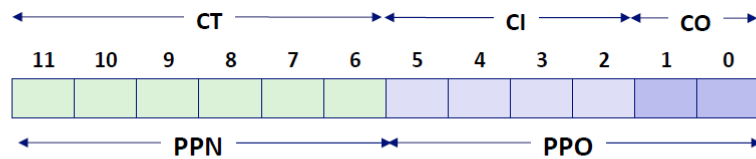
Only show first 16 entries (out of 256)

VPN	PPN	Valid
00	28	1
01	–	0
02	33	1
03	02	1
04	–	0
05	16	1
06	–	0
07	–	0

VPN	PPN	Valid
08	13	1
09	17	1
0A	09	1
0B	–	0
0C	–	0
0D	2D	1
0E	11	1
0F	0D	1

3. Simple Memory System Cache

- 16 lines, 4-byte block size
- Physically addressed
- Direct mapped



Idx	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
1	15	0	–	–	–	–
2	1B	1	00	02	04	08
3	36	0	–	–	–	–
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	–	–	–	–
7	16	1	11	C2	DF	03

Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	–	–	–	–
A	2D	1	93	15	DA	3B
B	0B	0	–	–	–	–
C	12	0	–	–	–	–
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	–	–	–	–

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

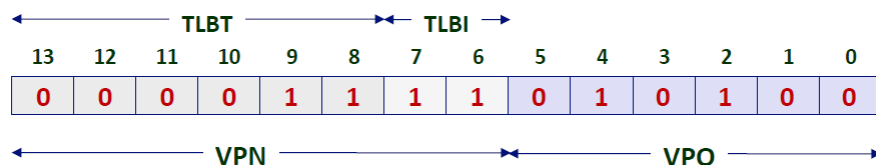
7

Cache一共16个块，块大小为4B，组织成直接映射，则E路的E=1，组数=块个数=16，set index位数= $\log_2 16 = 4$ ，block offset（块内地址）位数= $\log_2 4 = 2$ ，而总地址位数=PA位数=12，可以算出tag位数=12-4-2=6（图中为2位16进制数）

翻译的例子：

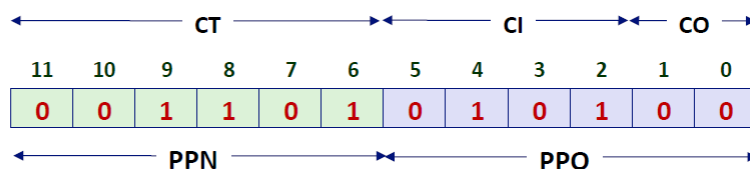
Address Translation Example #1

Virtual Address: 0x03D4



VPN 0x0F TLBI 0x3 TLBT 0x03 TLB Hit? Y Page Fault? N PPN: 0x0D

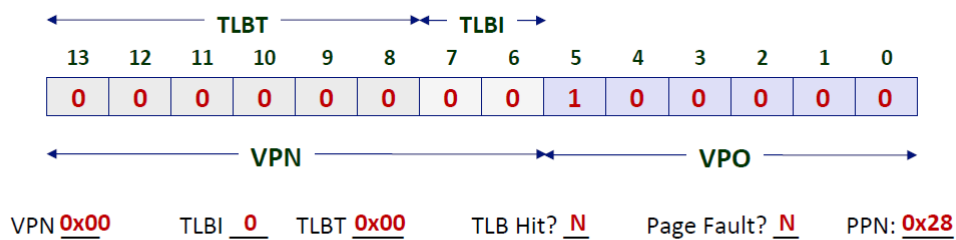
Physical Address



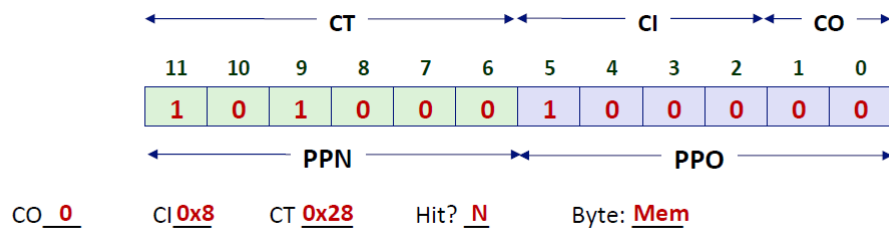
CO 0 CI 0x5 CT 0x0D Hit? Y Byte: 0x36

Address Translation Example #2

Virtual Address: 0x0020

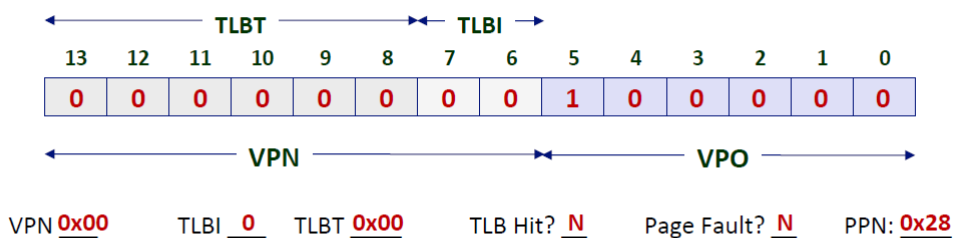


Physical Address

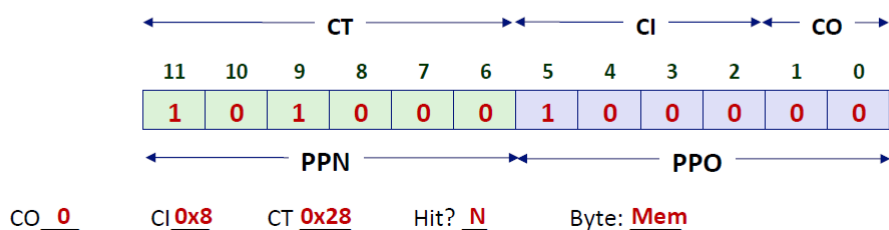


Address Translation Example #3

Virtual Address: 0x0020



Physical Address



35 外存储器

外存是**非易失性存储**，包括磁盘（disk）、磁带、光盘、SSD。

用途：存放CPU不立即使用的信息，需要的时候再调入主存。

特点：慢、**串行访问**（而非随机）、访问粒度大（单位是block）、掉电后信息不丢失。

磁盘 disk

磁盘访问过程

(物理上)

- 寻道 (seek) : 小车找到同心圆, 即磁道 (track)
- 寻找扇区 (rotate) : 等待磁道旋转到要读的扇区 (sector), 平均是旋转半周的时间 = $1 \text{ min} / \text{转速(rpm)} / 2$
- 数据传输 (io) : 数据读写的时间 = 数据大小 / 传输速度
- 【注意还有磁盘控制器延迟】如果串行访问就每次都要叠加, 因此改进方法可以是并行读多个层面 (platter), 或者并行读多个磁盘 (并行IO→RAID)。

思考: 如何将文件分布在扇区中使访问速度加快? ——应该和ppt中提到的页面一样, 放在**相邻扇区**, 避免**额外的寻道开销**。

(逻辑上) 缺页引起, 内存 (=主存) 中没有所需页, 要从外存 (通常是硬盘) 中调入。

- CPU给地址, 需要访问某存储单元;
- 并行进行TLB、cache查找, TLB没有找到, 停止并行查找, 通知OS处理;
- OS检查页表, 发现该页不在内存中, 需要从硬盘调入;
- 选择被换出的牺牲页,
 - 若是脏页, 则需写回, 写回时OS申请I/O总线, 发送写命令, 传回数据; I/O控制器会控制磁盘进行访问然后写入扇区。
 - 然后需要调入新页, 也是申请I/O,, 磁盘申请I/O总线, 得到授权后将数据通过总线送到内存。

可靠性 & 可用性

可靠性 (Reliability?) : 设备不出错。增加硬件冗余会使得数据容易出错, 可靠性降低。(但是后来RAID又说冗余能提高可靠性, 可能是之前某次课件讲到的Dependability via Redundancy, 后来问了老师, 老师说不管了(。))

可用性 (Availability) : 系统正常运行。方法可以是增加硬件冗余 (并行IO?) 。

——可用性的提高可能带来可靠性的降低: 冗余可以带来可用性提高, 但多个设备发生故障的概率也提高了, 故可靠性降低了。

GB/T3187-97对可用性的定义: 在要求的外部资源得到保证的前提下, 产品在规定的条件下和规定的时刻或时间区间内处于可执行规定功能状态的能力。

RAID

术语 (terminology)

Striping (分带) : 有n个驱动器 (drive) 就可以分n个带, 带的大小可以是byte / block level.

Striping can be done at byte level or block level. Byte-level striping means that each file is split up into parts one byte in size. Using the same 4 disk array as an example, the first byte would be written to the first drive, the second byte to the second drive and so on, until the fifth byte is then written to the first drive again and the whole process starts over.

In the case of block level striping, the above table can simply be repeated with the word 'block' substituted for 'byte'. Each file is split into parts one block in size, which is 512 bytes by default, but can be specified otherwise. The size of this block is commonly referred to as stripe size. If a file is smaller than the stripe size, it simply gets stored on a single disk.

Mirroring (镜像)

Parity (校验)

RAID分类

RAID-0: block-level striping with no redundancy

Disk 1	Disk 2	Disk 3	Disk 4
File 1, block 1	File 1, block 2	File 1, block 3	File 1, block 4
File 1, block 5	File 1, block 6	File 1, block 7	File 2, block 1
File 1, block 2	File 3, block 1	File 3, block 2	File 4 block 1
File 4, block 2	File 4, block 3		

因为文件可以被分在不同磁盘 (disk) 中, 所以对于随机读写很友好, 对于大文件读写也很不错。

RAID-1: Disk Mirroring with no striping

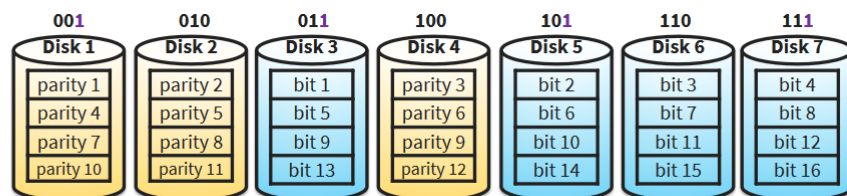
Disk 1 (data)	Disk 2 (mirror)
File 1, block 1	File 1, block 1
File 1, block 2	File 1, block 2
File 1, block 3	File 1, block 3
File 2, block 1	File 2, block 1
File 2, block 2	File 2, block 2
File 3, block 1	File 3, block 1
File 4, block 1	File 4, block 1
File 4, block 2	File 4, block 2
File 4, block 3	File 4, block 3

RAID-2: bit-level striping with ECC codes

parity 1 = $3 \oplus 5 \oplus 7$ (all disks whose # has 1 in LSB, xx1)

parity 2 = $3 \oplus 6 \oplus 7$ (all disks whose # has 1 in 2nd bit, x1x)

parity 4 = $5 \oplus 6 \oplus 7$ (all disks whose # has 1 in MSB, 1xx)



多了校验位, 可以纠正1位错, 但因为是bit-level分带, 一个字被分散了, 7个驱动器 (7个是因为海明码) 必须同步运动。controller很复杂, 因此代价昂贵。

RAID-3 & RAID-4: striping with a dedicated parity disk

RAID-3 是 **byte-level striping** + parity disk (驱动器要严格同步), 所以小文件读写也是在不同 drive上进行的。

Disk 1	Disk 2	Disk 3	Disk 4
File 1, byte 1	File 1, byte 2	File 1, byte 3	Parity
File 1, byte 4	File 2, byte 1	File 2, byte 2	Parity
File 3, byte 1	File 4, byte 1	File 4, byte 2	Parity
File 4, byte 3	File 4, byte 4	File 4, byte 5	Parity

好处: 增加了一个校验盘, 可以解决一个驱动器的错误; 顺序读取的性能比RAID-5好。

坏处: 因为是byte level striping, 对随机读写不友好; 校验盘负载沉重, 成为性能瓶颈。

RAID3 is unique in that it uses byte level striping as opposed to block level striping. This means that every file is split up amongst the striped drives equally, byte-by-byte, whereas files smaller than the block size in RAID-5 or RAID-0 are stored on a single drive. This feature of RAID3 is what gives it the added sequential read performance, as all files, no matter how small, are read from every drive.

RAID-4是 **block-level striping** + parity disk, 对磁盘访问有了更好的空间局部性, 也不需要驱动器同步, 但校验盘损坏得更快了。

Lesser Loved RAID5

RAID-2: bit-level striping with ECC codes



- 7 disk arms synchronized and move in unison
- Complicated controller (and hence very unpopular)
- Tolerates 1 error with no performance degradation

RAID-3: byte-level striping + parity disk

- read accesses all data disks
- write accesses all data disks + parity disk
- On disk failure: read parity disk, compute missing data



RAID-4: block-level striping + parity disk

- + better spatial locality for disk access
- parity disk is write bottleneck and wears out faster



<http://www.cs.cornell.edu/courses/cs4410/2017sp/schedule/slides/12.disks-raid.pdf>

<http://www.cs.cornell.edu/courses/cs4410/2020fa/schedule/slides/10-disks-raid.pdf>

RAID-5: block-level striping with distributed parity information

Disk 1	Disk 2	Disk 3	Disk 4
File 1, block 1	File 1, block 2	File 1, block 3	Parity
File 1, block 4	File 2, block 1	Parity	File 2, block 2
File 3, block 1	Parity	File 4, block 1	File 4, block 2
Parity	File 4, block 3	File 4, block 1	File 4, block 2

旋转校验 + block-level分带, 减少校验盘负载。

RAID 5: Rotating Parity w/Striping



	Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
Stripe 0	Strip (0,0) Parity (0,0,0) Data Block 1 Parity (2,0,0) Parity (3,0,0)	Strip (1,0) Data Block 0 Data Block 1 Data Block 2 Data Block 3	Strip (2,0) Data Block 4 Data Block 5 Data Block 6 Data Block 7	Strip (3,0) Data Block 8 Data Block 9 Data Block 10 Data Block 11	Strip (4,0) Data Block 12 Data Block 13 Data Block 14 Data Block 15
Stripe 1	Strip (0,1) Data Block 16 Data Block 17 Data Block 18 Data Block 19	Strip (1,1) Parity (0,1,1) Parity (1,1,1) Parity (2,1,1) Parity (3,1,1)	Strip (2,1) Data Block 20 Data Block 21 Data Block 22 Data Block 23	Strip (3,1) Data Block 24 Data Block 25 Data Block 26 Data Block 27	Strip (4,1) Data Block 28 Data Block 29 Data Block 30 Data Block 31
Stripe 2	Strip (0,2) Data Block 32 Data Block 33 Data Block 34 Data Block 35	Strip (1,2) Data Block 36 Data Block 37 Data Block 38 Data Block 39	Strip (2,2) Parity (0,2,2) Parity (1,2,2) Parity (2,2,2) Parity (3,2,2)	Strip (3,2) Data Block 40 Data Block 41 Data Block 42 Data Block 43	Strip (4,2) Data Block 44 Data Block 45 Data Block 46 Data Block 46
	⋮	⋮	⋮	⋮	⋮

RAID-6: block-level striping with double distributed parity information

比RAID-5多了一个校验盘，2个校验盘分别以不同方式进行校验，所以又称作二维校验。可以纠正2个盘上的错误。

Disk 1	Disk 2	Disk 3	Disk 4	Disk 5
File 1, block 1	File 1, block 2	File 1, block 3	Parity	Parity
File 1, block 4	File 2, block 1	Parity	Parity	File 2, block 2
File 3, block 1	Parity	Parity	File 4, block 1	File 4, block 2
Parity	Parity	File 4, block 3	File 4, block 4	File 4, block 5

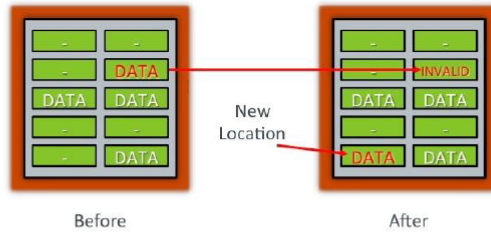
固态硬盘 (SSD)

固态硬盘 (Solid State Drives)：由控制单元、存储单元 (FLASH+DRAM芯片) 组成，用固态电子存储芯片阵列制成。

- 控制单元：FTL完成上层**逻辑地址**→底层**物理地址**的转换、**磨损均衡**。
- 存储单元：按**block有限次擦除**，按**页进行读写**，组织为package-die-plane-block-page

SSD的写入：

SSD的写入



- ▶ 与磁盘不一样，不会写入到原来的page，写入之前需要进行擦除操作
- ▶ 写入的时候不会在原来的page中写入，会在一个新的页面（可以在同一个块，也可以在不同的块，可能不同的plane，甚至不同的die上面）
- ▶ 这样，需要维护上层管理软件的逻辑地址和底层的物理地址之间的映射关系
- ▶ 这个工作交给FTL层来完成

SSD的垃圾收集：按**block**收集，先把block上的live page挪到其他地方，然后擦除整个block，完成垃圾回收。

36 RISC-V的系统模式

RISC-V的特权级模式

用户模式（执行用户应用程序）、**机器模式**（最高模式，直接接触硬件）、**监管者模式**（OS，只有实现了才能处理supervisor interruption和page fault）

权限：CPU大部分时间运行在低权限模式，处理中断异常时会将控制权交给更高的权限模式。

特权指令：很少，基本上通过【控制状态寄存器】（CSR, Constrol State Register）来实现。

硬件线程hart

异常分类:

- 同步异常，如ecall的环境调用异常（可以实现应用程序通过系统调用从OS中获得服务）、ebreak的断点异常（可以实现断点调试）、ID发现的非法指令异常、EX或者MEM发现的非对齐地址异常
- 异步异常即中断，中断源（来源）有**软件中断、时钟中断、外部中断**

CSR: mtvec, mepc, mcause, mie(目前能处理的中断), mip(正在处理的中断), mtval, mscratch, mstatus

- `mstatus.MIE` **置1时才会产生中断**，0会屏蔽所有中断；发生异常时会把 `mstatus.MIE` 保存在 `mstatus.MPIE`，然后 `mstatus.MIE` 置0以禁用中断（注意是 `mstatus` 寄存器中的 `MIE` 域，不是 `mie` 寄存器）
- 发生异常时的其他操作：`mepc <= 异常指令的pc`，`pc <= mtvec`；`mcause <= 异常原因`，`mtval <= 出错地址or so`，`mstatus.MPP <= 发生异常之前的权限模式`，`权限模式 <= M`
- 其他专门用于CSR读写的操作：`csrrw`等，基本都是同时读写的原子操作

wfi: 通知CPU进入低功耗模式, 直到使能有效的中断到来 (mie & mip != 0)

用户模式:

- 进程隔离，用户模式拒绝执行特权指令 / 访问CSR => 非法指令异常

- 物理内存保护，可以
 - **M模式指定U模式可以访问的内存地址**，维护PMPAddr和cfg寄存器。【优点】成本低，【缺点】只能支持固定数量的内存区域→可扩展性差，而且必须在物理存储中连续→碎片化，不能有效支持辅助内存（外存）的使用
 - **S模式进行页内存管理，支持虚拟内存**，注意S模式不能使用M模式用的CSR (mstatus) 和指令 (mret等)，而且受到PMP的限制
 - 启动分页，分页命名是SvX (X是以位为单位的VA长度，如Sv32和Sv39等)，此时大多数地址都是VA
 - 页表完成VA→PA，表项上有权限位指示哪些权限模式通过何种访问可以操作这个页
 - 访问未被映射的页 / 访问权限不足 → 抛出page fault exception
 - **satp** 寄存器 (Supervisor Address Translation & Protection) 进行监管者地址转换和保护
 - 当M模式的程序第一次进入S模式时会将0写入 **satp**，禁用分页 (所以正常实现了页表的kernel是不会出错的)
 - 然后S模式的程序在初始化页表之后 (可以用虚拟地址了) 会再次写 **satp** 寄存器，开启分页
 - ASID (Address Space Identifier) 地址空间标识符，用于降低上下文切换开销
 - TLB: 减少访问页表的开销，作为页表的cache。【问题】如果OS修改了页表，之前的TLB就不可用了，需要通过 **sfence.vma** 通知CPU刷新TLB的表项 (rs1指示某VA对应的转换, rs2指示被修改页表的ASID，若均为x0则整个TLB刷新)

系统编程的两大主题：**异常和中断处理、内存管理**

41 输入输出系统和设备

管理外部设备需要解决的问题：

- 控制方式，即CPU如何控制外设的输入/输出（本讲为输入输出方式，有5种）
- 传输方式，即使用什么传输通道传输地址、数据、控制等信息，其方式和速率等（下一讲总线、再下一讲接口）
- 数据识别和转换，本门课程不讲（信原内容？）

输入输出方式：

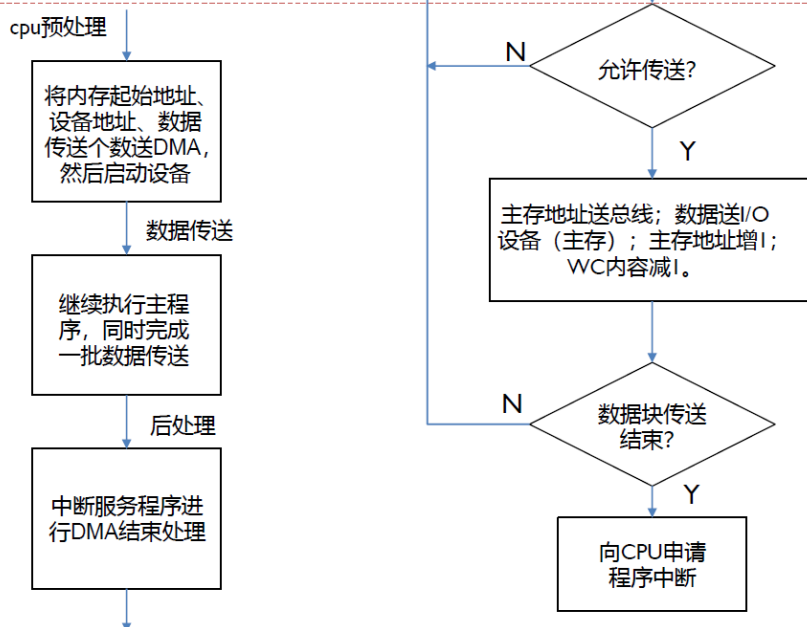
- **程序直接控制**，通过轮询。【特点】成本低、效率低、严重占用CPU资源
- **程序中断方式**，CPU和外设并行工作，外设发起请求，CPU收到中断，停止之前的程序进行相应。【特点】提高CPU效率，可以同时管理多个外设，但是对CPU干扰较大，适用于传输速度不高、传输量不大的情况。此时的CPU中断——
 - 中断源是外设 (I/O设备)
 - 中断优先级决定了响应的顺序
 - 中断的完整过程：
 - (1)中断请求：中断源设备设置中断触发器。每个中断源有一个中断触发器。同时可设置一个中断屏蔽触发器。
 - (2)中断响应：响应条件（允许中断、当前指令结束、优先级）、响应实现（硬件实现的中断隐指令，保存断点）
 - (3)中断处理：

- (保存上下文) 保存现场信息、运行中断服务程序、中断返回关中断 → 保存断点 →
 - (处理) 判中断源转中断服务 → 开中断 → 执行中断服务程序 →
 - (恢复上下文) 关中断 → 恢复断点 → 开中断 → 返回断点
- 直接存储访问 (DMA, Direct Memory Access) 外设和主存储器直接相连, 是为外设专设的硬件, 用于高速外设和主存之间【成组传送】数据。传送开始前 & 结束后都通过程序 or 中断方式使CPU对DMA进行预处理和后处理, 但传送时CPU是不被影响的, 可以继续执行程序。
 - DMA工作方式: 独占总线 / 周期窃取 (因为DMA会占据总线, 所以外设不能太慢), 42 bus课件还提到DMA与CPU交替访内的方式使用内存总线

DMA使用内存总线的方式

- ▶ 独占使用: 当外设要求传送一批数据时, 由DMA控制器发一个信号给CPU。DMA控制器获得总线控制权后, 开始进行数据传送。一批数据传送完毕后, DMA控制器通知CPU可以使用内存, 并把总线控制权交还给CPU。
- ▶ 周期挪用 (周期窃取): 当I/O设备没有DMA请求时, CPU按程序要求访问内存: 一旦I/O设备有DMA请求, 则I/O设备挪用一個或几个周期。(随时, 一旦冲突, DMA优先)
- ▶ DMA与CPU交替访内: 一个CPU周期可分为2个周期, 一个专供DMA控制器访内, 另一个专供CPU访内。不需要总线使用权的申请、建立和归还过程。

◦ DMA数据传送过程



- 特点: 与设备一对一服务 (多DMA控制器同时工作可能发生冲突), 对CPU干扰适中, 无法适用大量高速设备的管理
- 【问题】存储页可能有的使用VA有的用PA; Cache一致性问题
- 通道控制方式
 - 特点: 一对多的连接关系, 适应不同速度、不同种类的外部设备, 可并行工作
 - 功能: 根据CPU的要求选择某一外设与之连接, 控制地址、数据交换以及检查外设状态

通道控制方式

是对DMA控制方式的发展，类似一个弱的CPU

- I/O通道是计算机系统中代替CPU管理控制外设的独立部件，是一种能执行有限I/O指令集合——通道命令的I/O处理机。

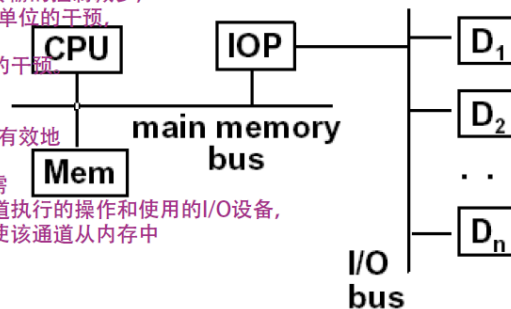
通道本质上是一个简单的处理器，专门负责输入、输出控制，具有执行I/O指令的能力，并通过执行通道I/O程序来控制I/O操作

- 一对多的连接关系

- 适应不同速度、不同种类的外部设备，可并行工作

①进一步使CPU参与到数据传输的控制减少，即把对一个数据块的读/写为单位的干预，减少为对一组数据块的读/写及有关的控制和管理为单位的干预。

②同时，实现CPU、通道和I/O设备的并行操作，从而更有效地提高整个系统的资源利用率。在通道控制方式中，CPU只需发出启动指令，指出要求通道执行的操作和使用的I/O设备，该指令就可以启动通道，并使该通道从内存中调出相应的通道程序执行。



• 外围处理机

外围处理机

用一个功能较弱的计算机管理 I/O，彻底解放 CPU

- 通道型处理机

- 共享内存

- 外围处理机

- 通用计算机

- 独立完成输入/输出功能

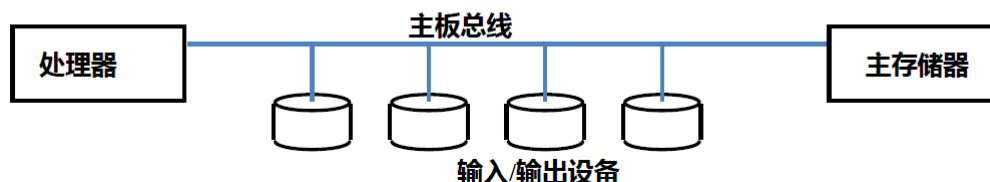
- 通过通道方式与主机进行交互

42 总线

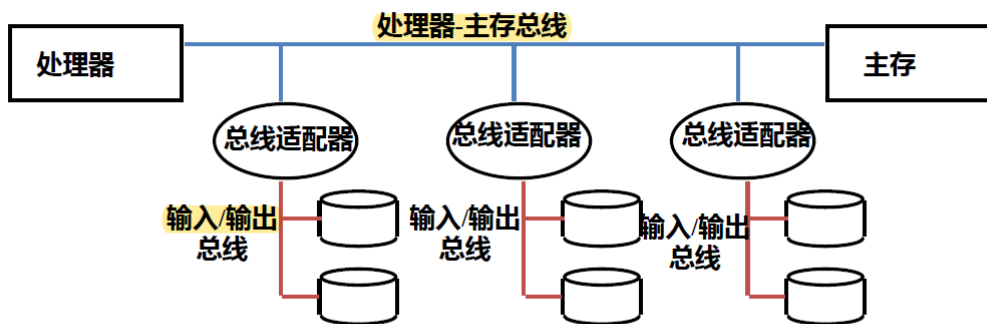
总线：计算机多个子系统共享的信息通道，需要关注控制、地址、数据三类信号之间的关系

使用总线的好处：统一标准、降低成本、简化设计；【缺点】总线容易成为瓶颈，限制吞吐量

单总线计算机：主板总线



双总线系统



▶ 输入/输出总线通过适配器和处理器-主存总线相连：

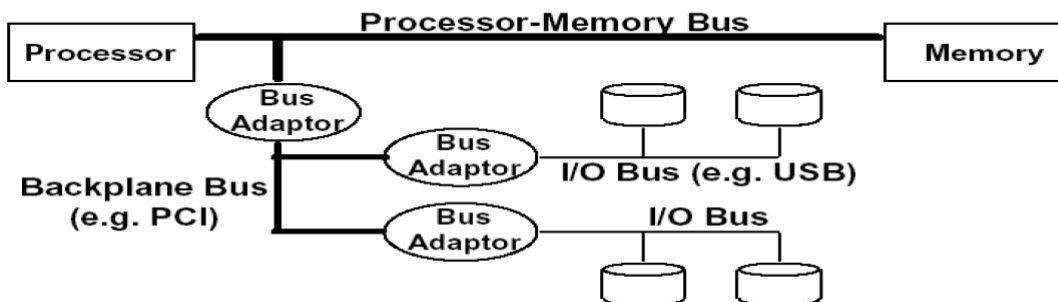
- ▶ 处理器-主存总线：主要用于处理器和主存储器之间的通信
- ▶ 输入/输出总线：为输入/输出设备提供信息

1. 内存总线只连memory和bus adapter（北桥芯片）

2. Bus adapter连主板总线

三总线系统

3. 主板总线再通过bus adapter（南桥芯片）连外设



▶ 主板总线连接到处理器-主存总线

▶ 处理器-主存总线主要用于处理器和主存之间数据交换

▶ I/O总线连接到主板总线

▶ 优点

▶ 大大减少处理器-主存总线负载

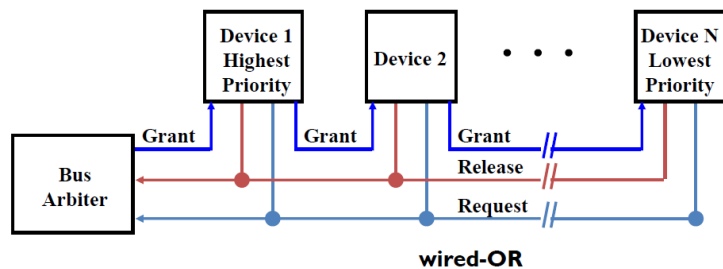
总线类型

- 处理器-主存总线（主存储器专用）
- 输入/输出总线（行业标准）
- 主板总线（行业标准或专门设计）

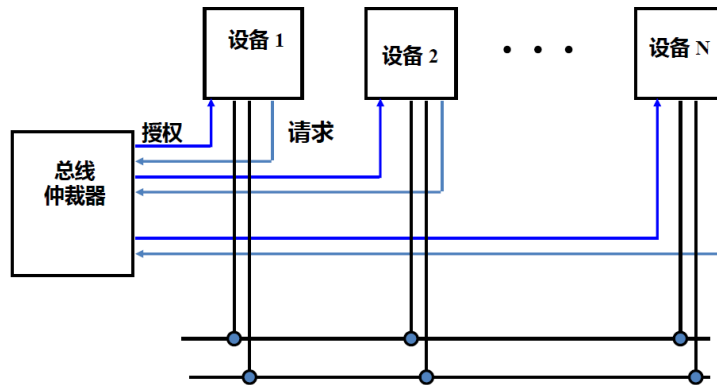
总线的一般组成：控制线（信号和信息类型）、数据线（数据和地址）

总线事务：发起命令，传输数据

- 主设备：发起；一群主设备都需要发起命令时，仲裁获得总线使用权，得到授权的那个进行使用，用完了通知仲裁器
 - 总线仲裁方式：
 - 集中仲裁：
 - 菊链仲裁（简单但可能不够公平）



■ 集中平行仲裁（常用）



■ 分布仲裁

○ 其他总线仲裁的划分方式：优先级仲裁 / 轮循仲裁

- 从设备：响应

总线工作方式（应该是？）：

- 同步总线，根据时钟信号 => 为防止信号扭曲，总线距离需要比较短
- 异步总线，距离可以长，使用握手协议

增加总线带宽的方式：

- 增加总线的宽度，可以增加每个周期传送数据的量 => 成本提高
- 分离地址总线 and 数据总线，可以同时传送 => 成本提高
- 采用成组传送方式 => 复杂度提高，延长了后续请求的等待时间

多主设备总线提高事务数量（？）：仲裁重叠，总线占用，地址、数据传送重叠

【例子】PCI总线

PCI 总线的读/写事务

- ▶ 所有信号在时钟正边沿采样
- ▶ 集中平行仲裁 把总线仲裁分配给某一主设备
 - ▶ 和上一事务重叠
- ▶ 所有事务可无限制成组传送
- ▶ 地址段起始于 FRAME#信号有效
- ▶ 第一时钟周期主设备发出cmd和address
- ▶ 数据传送
 - ▶ 当主设备准备好传输数据，主设备发出IRDY# 信号 Initiator
 - ▶ 从设备准备好传输数据，发出TRDY#信号 target
 - ▶ 上述两个信号均有效时的时钟上升沿开始传送数据
- ▶ 主设备准备结束数据传送时，将FRAME#信号失效

①FRAME#有效：开始读，主设备指定地址AD，发送读命令。
②主设备释放AD，指定有效位，置IRDY，准备好接收数据；
从设备被选中后置DEVSEL，读出数据到AD，置TRDY通知主设备接收。
③主设备在周期4读到（可能更迟，如果从设备慢要插入等待周期），
FRAME#变化通知从设备结束传输。从设备拉高DEVSEL和TRDY，总线回到空闲状态。

PCI读

- ▶ 总线主设备得到授权后，将FRAME#置为有效，开始读事务。并通过AD发送要读的地址，C/BE#发送读命令
- ▶ 从设备从AD上识别是否被选中
- ▶ 主设备释放对AD的控制，同时，在C/BE#上给出AD上哪些位是有用的（1~4Bytes）。并置IRDY#为有效，表示已准备好，可以接收数据。
- ▶ 被选中的从设备置DEVSEL信号，表示已收到命令并可响应。将读出的数据送AD，并置TRDY#通知主设备接收。

43 接口电路和外部设备

接口：总线和外部设备的连接，基本功能有

- 使主机能根据设备编号找到外设
- 使主机能控制设备并与之通信
- 提供主机和设备之间的数据缓冲机制

USB采用同步传输方式，因此USB帧还设置有同步帧，用于同步。

参考资料

刘卫东、陈康、陆游游老师编写的计算机组成原理课件

byz的笔记&我自己的笔记

lxq的计组复习提纲

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition (《深入理解计算机系统》, 可以从官方网站上获取全套CMU课件及lab) (Cache和VM讲得非常清楚)

David Patterson and John Hennessy, Computer Organization and Design, Third Edition (《计算机组成与设计: 硬件软件接口》) (参考了里面的IO章节)

CS 4410 by Professors Sirer, Bracy, Agarwal, George, and Van Renesse, Operating Systems (Spring 2017) (可以在Cornell University官网上获取全套课件, 但是是操作系统的课.....搜索RAID找到的)

Chiu @ 2021.1.10